Agentic Edge–Cloud Orchestration for Resource-Efficient Daily Food Inventory Monitoring with Egocentric Video

Anonymous Author(s)

Abstract

Daily food inventory monitoring can help households avoid redundant purchases, manage food safety, and plan meals around existing food. Egocentric video from smartglasses creates new opportunities for this task by capturing everyday kitchen activities, where a system tracks which food items are used and how much remains in each package or container. Cloud Vision-Language Models (VLMs) can reason over such visual evidence, but whole-video inference is too costly for recurring deployment. At 1 fps, processing one hour of daily video with Gemini 3.1 Pro costs over $100 per household per month, while long irrelevant context can distract the model from accurate inventory-change reasoning. Our key insight is that daily inventory updates are determined by brief, sparsely distributed evidence moments within long recordings. We present *InvAgent*, an agentic edge–cloud orchestration system for resource-efficient daily food inventory monitoring with egocentric video. *InvAgent* uses an edge context builder to extract item-interaction timeline context with edge-deployable lightweight models, an agentic Large Language Model (LLM)-based planner to select amount-evidence windows and replan for unresolved items, and a cloud VLM reasoner to examine only selected frames and estimate each item's starting amount, amount consumed, and remaining amount when visible. We collect a benchmark dataset of 39 h of natural egocentric kitchen video from 10 participants across 7 households with measured food-amount changes. Compared with whole-video Gemini 3.1 Pro inference, *InvAgent* reduces input tokens by 9.9× and projected monthly cloud cost by 5.1×, while improving amount-aware inventory F1 with modest edge overhead: 0.45 s amortized latency per sampled frame on a Jetson Orin.

1 Introduction

Daily food inventory monitoring requires a system to maintain an up-to-date record of which household food items are available and how much remains, so the system can answer questions such as "Do I still have eggs at home?" or "Is my milk expired?" Better awareness of current stock and logged purchase dates can help households avoid redundant purchases, plan meals around existing food, and flag forgotten perishables approaching typical use-by periods, mitigating a practical food-safety concern: missed use-by dates are a known household foodborne-illness risk factor [6, 7]. In the United States, 30–40% of the food supply is wasted, costing an average family of four nearly $1,500 per year [48, 49], while foodborne illness affects an estimated 48 million people annually and causes 128,000 hospitalizations and 3,000 deaths [1].

Consumer smartglasses make it increasingly practical to capture hands-free egocentric video of kitchen interactions, including retrieval, opening, transfer, put-back, and discard actions that determine how much food remains [19, 20]. Large VLMs, meanwhile,

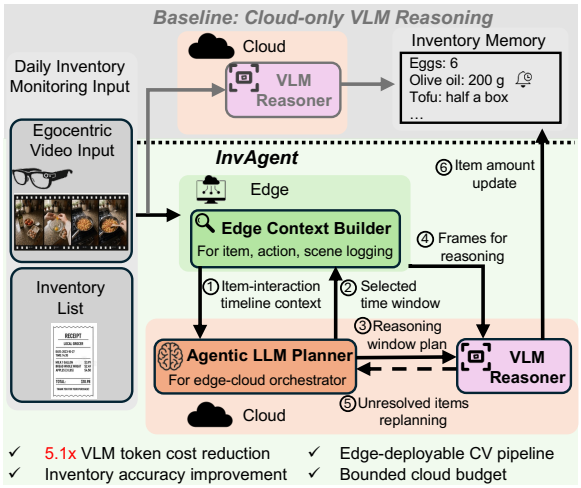


Figure 1: *InvAgent* Overview. The baseline sends the whole video to a cloud VLM to update remaining amount of food. In *InvAgent*, the Edge Context Builder converts full egocentric video into compact item-interaction timeline context, enabling the cloud Agentic LLM Planner to select VLM reasoning windows and replan for unresolved items.

make this video actionable by recognizing diverse food items, inferring package and container state, and answering natural-language inventory queries [45, 46, 54].

A straightforward approach is to continuously stream egocentric video from smartglasses to a cloud VLM for summarizing the food inventory state. However, this design is difficult to deploy for recurring household use. *First*, continuous VLM reasoning incurs inference cost that scales linearly with video length. *Second*, VLM accuracy can degrade as the input context grows, because irrelevant visual content can distract the model from the sparse evidence needed for inventory updates even when the input remains within the model's context window [41, 50]. Our preliminary study shows that, at Gemini 3.1 Pro May 2026 paid-tier rates [12], processing one hour of 1 fps egocentric video costs $3.66, exceeding $100 per household per month with only one hour of daily logging [47]. Recent agentic long-video understanding systems reduce VLM reasoning load by using LLM/VLM loops to retrieve or revisit selected segments [9, 39, 42, 52, 53, 56, 57, 59]. However, these systems are designed to answer individual video queries, not to optimize the cost–accuracy trade-off of a recurring multi-item state-update task.

To address this bottleneck, we make two key observations. *(1)* Only a small subset of activities can change the remaining amount of a food item, such as pouring, scooping, removing a portion, discarding, or putting the package back after use. *(2)* VLM reasoning is only needed for the sparse inventory amount evidences, where they can be used to estimate the starting amount, transferred amount,

or remaining amount. The challenge is to localize these sparse moments without sending the entire egocentric video to the cloud for reasoning. This motivates edge–cloud orchestration: an edge device provides a compact, low-cost representation of the full recording, enabling the cloud to perform expensive VLM reasoning only on selected amount-evidence windows.

We propose *InvAgent*, an agentic edge–cloud orchestration framework for resource-efficient daily food inventory monitoring using egocentric video. Our goal is to *exploit sparse, localizable item-use structure at the edge to guide cloud-side VLM reasoning, reducing VLM input-token consumption and cloud cost while preserving numeric inventory-update quality.* As shown in Fig. 1, *InvAgent* treats smartglasses as capture-only devices: they record egocentric video, while the user-provided inventory list enters *InvAgent* as a separate input. Before cloud reasoning, *InvAgent* uses an Edge Context Builder to construct the item-interaction timeline context. The Agentic LLM Planner then uses this context to select the essential time windows for VLM reasoning. The VLM Reasoner reads only the selected frames and returns grounded starting amount, amount used, and remaining amount readings when visible. It also reports unresolved items back to the Agentic LLM Planner for item-specific replanning within a bounded round budget. Making this architecture work requires addressing two major design challenges.

How can the edge produce a compact, planner-usable item-interaction timeline context under computation constraints? InvAgent uses a household edge endpoint, such as a Jetson-class device [27] or a local GPU server, to build a compact contextual representation before cloud VLM reasoning. The challenge is that the edge must process the full egocentric recording efficiently while preserving temporal and semantic structure for cloud-side planning. To this end, the Edge Context Builder applies edge-deployable frame-level CV models to 1 fps sampled video and extracts per-frame spatial context, such as candidate item detections and item-interaction cues. It then uses a Timeline Context Aggregator to merge per-frame context over time, link temporally related interactions, and produce a compact item-interaction timeline context. This representation gives the Agentic LLM Planner a structured view of the full session, enabling it to select candidate amount-evidence windows without sending the full video to the cloud VLM. The CV models used by the Edge Context Builder are replaceable, allowing *InvAgent* to adapt to different edge endpoints and balance local computation cost with end-to-end inventory monitoring cost and accuracy.

How can the cloud planner balance evidence coverage and VLM cost when selecting candidate amount-evidence windows from imperfect edge context? InvAgent uses an Agentic LLM Planner to select VLM reasoning windows from the item-interaction timeline context. However, edge-generated timeline context is imperfect: it may identify candidate item-use moments without revealing whether a window contains the pre-use package state, transferred amount, or post-use remaining amount needed for a numeric update. A one-shot planner may thus select incomplete evidence, causing the VLM Reasoner to miss critical amount information and reducing inventory accuracy. To balance this trade-off, *InvAgent* adopts an agentic multi-round plan–observe–replan strategy. The planner first selects the candidate windows, and the VLM Reasoner examines those frames to produce structured amount readings and an evidence-sufficiency judgment for each item. If an item remains unresolved, the planner uses the VLM reasoning results and edge context to request additional item-specific windows. This loop improves evidence coverage while limiting additional cost by replanning only for unresolved items within a bounded round budget.

Because existing egocentric-video datasets lack package-level food-amount annotations, we collect a benchmark dataset of egocentric kitchen videos with measured food amounts before and after use. The dataset contains 38.97 hours of egocentric kitchen video from 10 participants across 7 households, covering 604 usage events and 254 tracked food item-instances. We implement the end-to-end *InvAgent* pipeline using an NVIDIA Jetson Orin as the edge platform and commercial cloud LLM/VLM APIs from the Gemini and GPT families as the planner and reasoner platforms. An anonymized package with code, prompts, and evaluation scripts is available for review.[1] We measure amount error as the absolute difference between predicted and ground-truth remaining amount, normalized by package capacity, and use amount-aware inventory F1 as the primary metric, which combines item-use detection with clipped remaining-amount quality while penalizing hallucinated uses. With the default Edge Context Builder, *InvAgent* uses 9.9× fewer input tokens and reduces projected monthly cloud cost by 5.1× compared with whole-video VLM inference, while improving amount-aware inventory F1 from 66.6% to 68.9%. Edge-model swaps expose a deployment tradeoff: the lightest profiled Jetson setup adds only ≈ 0.45 s overhead per sampled frame while preserving comparable inventory F1. Across all seven households, *InvAgent* reduces projected monthly cost by 68–92% and improves amount-aware inventory F1 in five of seven households. The cost-saving pattern also persists across Gemini Pro/Flash reasoner tiers, where the cheaper Flash reasoner trades accuracy for lower monthly cost.

In summary, we make the following contributions:

- We develop *InvAgent*, an end-to-end system for daily food inventory monitoring using egocentric video, targeting per-item use detection and package-level remaining-amount updates.
- We design an agentic edge–cloud architecture in which the Edge Context Builder builds the planner's initial context, and a bounded Agentic LLM Planner progressively requests VLM evidence only for unresolved amount readings.
- We collect and will release a benchmark dataset of 38.97 h of natural egocentric kitchen video from 10 participants across 7 households with measured package-level food-amount changes.
- We evaluate *InvAgent* end to end, showing that it reduces projected monthly cloud cost by 5.1× while improving amount-aware inventory F1 over whole-video VLM inference.

2 Task Definition and Motivation Study

In this section, we first define the task addressed by *InvAgent* and then present preliminary studies that motivate our design.

2.1 Task Definition

As shown in Fig. 1, the input to the inventory monitoring task is a natural, untrimmed egocentric recording of a kitchen activity session and a household inventory monitoring list. The monitoring list contains all tracked item instances that have not yet been depleted. Each entry stores the item's identity, reference image, package size,

[1] https://anonymous.4open.science/r/InvAgent-93B0/

and measurement unit, which can be obtained from digital grocery receipts and product databases. We estimate the *amount* of each item, which is the quantity that a household would naturally record: weight for fluids, granular foods, and other bulk items, and count for discrete items such as eggs, fruit, or individually packaged units. Given the video and monitoring list, the system will *(1) decide which items were used* and *(2) update the state of each used item with its remaining amount* from inventory amount evidence. Sec. 4 provides the full collection protocol and item/household breakdowns.

2.2 Motivation Preliminary Study

We first conduct preliminary studies with two baselines: *(1) whole-video VLM inference*, which takes an entire egocentric video session as input and directly outputs remaining-amount estimates for each used item. It represents a straightforward cloud-only approach by leveraging the VLM's long-context reasoning capability to understand the full kitchen activity session and infer inventory updates. *(2) human-annotated inventory-window baseline*, in which a human annotator identifies inventory-interaction intervals from the full video, and the remaining amount is estimated from these manually selected clips. Throughout this section, we ground each claim using a pilot dataset drawn from our full dataset (§4), consisting of 10 egocentric kitchen activity sessions, approximately 2.2 hours of video at 1 fps, 7,977 sampled frames, and 64 item usage events. Across these sessions, we annotated four types of inventory-interaction intervals for each tracked item: retrieval (taking an item from storage), access (opening or unsealing it), portion transfer (pouring, scooping, cutting, removing, or discarding food), and restocking (returning the item to storage). We summarize our insights as follows.

(1) Whole-video VLM inference cost an average more than $100 *per month for inventory monitoring.* We send the 1 fps egocentric video session to Gemini 3.1 Pro and ask the model to infer the final inventory state end to end. On our pilot, this baseline costs $3.66 per hour of recording at May 2026 paid-tier rates [12], resulting in over $100 per household per month with one hour of daily kitchen video logging. The dominant cost driver is the linear scaling of input tokens with video length [12, 30]. This finding shows that long-horizon egocentric video is too costly to be routinely processed by a cloud VLM end to end.

(2) Whole-video context buries amount evidence. The whole-video baseline also reveals a context-distraction problem: most frames in an egocentric kitchen session do not contain amount evidence. When a VLM receives the full session, the answer-bearing views are buried among hundreds of visually redundant or misleading frames, including unrelated kitchen activities, off-target object handling, hand occlusions, motion blur, and views of food after it has left the queried package, which no longer reveal the package's remaining amount. On our pilot dataset, Gemini 3.1 Pro detects only 53 of 64 usage events (82.8%) when given the whole-video context. In comparison, the human-annotated inventory-window baseline recovers 58 of 64 events (90.6%). This diagnostic suggests long video context can distract the VLM from the sparse views needed for numeric inventory updates.

(3) Inventory-interaction intervals are sparse in the whole video session. The design opportunity is that the moments needed to estimate an item's remaining amount are sparse but localizable. On our pilot dataset, these annotated inventory-interaction intervals account for only roughly 16% of total kitchen-video duration. The rest of the timeline includes food preparation around non-query items, cooking, waiting, washing dishes, and other activity not directly changing inventory state [32]. These observations shift the problem from recognizing food items to deciding where expensive visual reasoning should be spent: before any cloud VLM call, the system needs item-interaction timeline context: a compact text representation of which monitored items were interacted with and when.

Overall, the preliminary study shows that pure cloud VLM reasoning over long-horizon egocentric video is impractical for daily food inventory monitoring: its cost scales with video length, while amount evidence for accurate amount updates is sparse and localized. The key opportunity is to invoke VLM reasoning only on amount-evidence windows, such as pre-use package views, transfer actions, and post-use package-state views, like the *human-annotated inventory-window baseline*. The challenge is that these windows are not known in advance. Using the cloud VLM itself to search the full video would defeat the cost goal, while relying on the capture device for sustained full-session analysis would exceed practical battery, thermal, and memory constraints. This motivates *InvAgent* to introduce an edge computing component that uses edge-deployable models to provide full-session context for the cloud, enabling selective VLM reasoning on candidate amount-evidence windows while reducing cloud token cost and preserving inventory-update quality.

3 System Design

To achieve the goal discussed in the previous section, we design *InvAgent*, an agentic edge–cloud orchestration framework, as shown in Fig. 1. It contains three key components: *(1) Edge Context Builder.* Deployed on a household edge computing platform, such as an NVIDIA Jetson-class device [27] or a local GPU server, the Edge Context Builder scans the full egocentric recording and constructs compact item-interaction timeline context. This context provides a compact view of the session before any cloud VLM reasoning is invoked. *(2) Agentic LLM Planner.* The cloud-based Agentic LLM Planner takes the edge-generated item-interaction timeline context as input and identifies candidate amount-evidence windows for inventory updates. *(3) VLM Reasoner.* The VLM Reasoner performs detailed visual reasoning over only the selected amount-evidence windows to estimate each item's amount update. In addition to producing structured readings of starting amount, amount used, and remaining amount, it reports whether the selected frames provide sufficient amount evidence for each item. If amount evidence is insufficient, the Agentic LLM Planner replans and requests additional item-specific windows within a bounded round budget.

3.1 Edge Context Builder

The goal of Edge Context Builder is to provide compact, planner-usable item-interaction timeline context under computation constraints. To achieve that, we proposes to gather the context from three perspectives, as shown in Fig. 2.

Item Attributes. As discussed in Sec. 2.1, *InvAgent* assumes that the household inventory list is available from physical or digital records, together with each item's reference image. Each item is associated with three attributes: *amount unit*, *package capacity*, and

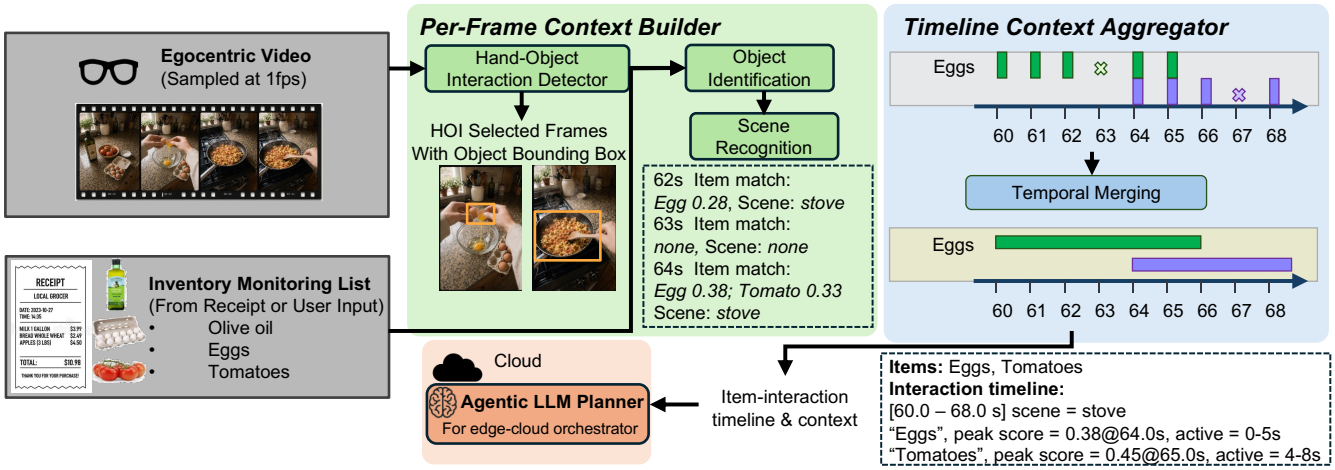


Figure 2: Edge Context Builder pipeline. Left: egocentric video and the inventory monitoring list provide sampled frames and registered item targets. Middle: the Per-Frame Context Builder localizes hand-object contact, assigns item-match scores, and tags scene context. Right: the Timeline Context Aggregator temporally merges per-frame hits into time-ordered item-interaction timeline context for the Agentic LLM Planner.

transparency tag. The amount unit specifies how the item's remaining amount is represented. We categorize items into four types: *fluid*, *granular*, *solid*, and *discrete*. Fluid, granular, and solid items are measured by their package-level remaining weight, while discrete items are measured by count. The package capacity defines the item's full-package amount, which is used to normalize remaining-amount estimates and compare predictions across items. The transparency tag indicates whether the package or container allows direct visual inspection of the remaining amount. For transparent packages, the planner can prioritize first-seen and last-seen package views, which may directly reveal the starting and remaining amount. For opaque packages, the planner must instead seek item-interaction evidence, such as transfer, scoop, pour, or before/after package-state views, to infer the amount change.

Per-Frame Context Builder. The first challenge for the Edge Context Builder is to scan the full egocentric video under edge compute constraints while preserving the cues needed for later cloud-side planning. A naive frame-level representation would either be too expensive to compute or too noisy for the planner to use directly. *InvAgent* therefore builds a compact, planner-usable per-frame context from 1 fps sampled video. We choose 1 fps because common inventory-changing actions, such as retrieval, opening, dispensing, and put-back, typically unfold over multiple seconds; this rate provides sufficient coverage of such actions without requiring dense video processing.

As shown in Fig. 2, the Per-Frame Context Builder extracts two key types of context. First, a hand–object interaction (HOI) detector identifies frames in which a package-level state change may occur and returns the interacted object bounding box, shown as the masked crop in Fig. 2. Second, an object identification model binds the interacted object to a known household inventory item by matching the object crop against registered item reference images using visual embeddings [10]. For each monitored item, this step outputs an item-match score: the similarity between the contacted crop embedding and that item's registered reference-image embeddings. When edge resources permit, *InvAgent* extracts additional context from each frame. In our current implementation, an open-vocabulary object detector identifies common kitchen scene contexts, i.e., storage, counter, and stove.

The output of Per-Frame Context Builder is a compact tag-based representation for each sampled frame. It records where a monitored item may have been handled, which inventory entry it likely matches, and what local scene context surrounds the interaction. This representation is the key for the planning stage: it turns raw egocentric frames into structured, inventory-grounded observations that a cloud planner can later use to select amount-evidence windows, without first sending the whole video to a cloud VLM. The CV models in this stage can be replaced to match the compute resources of the edge platform. By default, we use the following edge-deployable models: Hands23 [3] for hand–object interaction detector, DINOv3 [43] embeddings for object identification, and OWLv2 [23] for scene recognition. These model choices affect the accuracy and completeness of the item-interaction timeline context, which in turn influences downstream cloud-token cost and amount-update quality. We study how these choices affect end-to-end performance in Sec. 5.6.

Timeline Context Aggregator. The per-frame context described above provides detailed item-interaction observations, but it is not yet suitable as direct input to the Agentic LLM Planner. First, per-frame logs are dense: a single item use spans several seconds, so sampled frames can produce many repeated records for the same interaction. Second, item matches are intermittent because hands, utensils, motion blur, or partial package views can hide the registered item for a few frames. To test whether temporal grouping matters, we compare two edge-output representations on the pilot videos while keeping downstream processing fixed: raw per-frame logs and temporally merged item-interaction segments. Per-frame logs increase the cloud input by 3.8× (14.5K vs. 3.9K tokens per session) and reduce recall from 93.8% to 87.5%. Therefore, the edge

Algorithm 1 Timeline Context Aggregator

Input: times T, contact times H, item scores $m_i(t)$, scenes $s(t)$, items I; parameters $\tau, g, d_{\min}$
Output: item-interaction timeline context C_0
Operators: RUNS: maximal active ranges; $\textsc{CloseGaps}_g$: merge gaps $\leq g$; $\textsc{DropShort}_{d_{\min}}$: drop ranges $< d_{\min}$

for all $i \in I$ **do**
  $A_i \leftarrow \{t \in T \cap H : m_i(t) \geq \tau\}$
  $R_i \leftarrow \textsc{DropShort}_{d_{\min}}(\textsc{CloseGaps}_g(\textsc{Runs}(A_i, T)))$
  label each $r \in R_i$ with $\text{span}(r)$ and $t^{\star}_{i,r} = \arg\max_{t \in r} m_i(t)$
end for
$\mathcal{B} \leftarrow \textsc{ClusterOverlaps}(\bigcup_{i \in I} R_i)$
return $C_0 \leftarrow \textsc{RenderTimeline}(\mathcal{B}, s)$

output should group repeated detections into temporally coherent item-interaction segments before sending them to the cloud.

This design is based on the observation that common inventory-changing actions, such as retrieval, opening, dispensing, and put-back, typically span multiple seconds. Alg. 1 summarizes how the Timeline Context Aggregator converts contact detections and item scores into item-interaction timeline context. Thus, items handled in the same preparation interval appear in one timeline entry with per-item active spans, rather than as repeated per-frame hits. Each item-interaction segment records start and end time, dominant scene tag, active inventory items, a peak-match timestamp for each item, and the interval over which each item is visible during contact. The resulting item-interaction timeline context is a compact, text-only representation consumed by the Agentic LLM Planner. It provides a session-level hypothesis of where monitored items may have changed state, while preserving enough timing, scene, and item-match evidence for the planner to select amount-evidence windows.

3.2 Agentic LLM Planner

After the edge constructs the item-interaction timeline context, *InvAgent* uploads this compact text context to the cloud. The role of the Agentic LLM Planner is to convert this edge-generated timeline context into a small set of candidate amount-evidence windows.

Our key insight is that *item-interaction segments are not necessarily amount-evidence windows*. For a given item, the edge output is a sequence of short interaction segments, typically 5–15 s each. These ordered segments describe how an item is used, such as retrieval from storage, opening or unpacking, portion transfer, views of food after it leaves the package, and put-back or final visibility. However, the evidence needed for amount estimation usually appears in only a small part of this sequence. For example, in Fig. 3, tomatoes may first appear in a brief bag view, then on a cutting board, and later in a pan. The cutting-board views help confirm that tomatoes were used, but the brief bag view and pan view can not reveal how much remains in the bag. This means that an interaction segment can be visually rich while still missing the answer-bearing frames needed for a numeric remaining-amount update. To quantify this, we use manual annotated intervals in the pilot dataset from Sec. 2.2, where a used item's post-use remaining amount is directly visible. The Edge Context Builder retains 46% of session frames, but only 19.5% of those retained frames fall within remaining-state evidence intervals. This mismatch motivates an Agentic LLM Planner that selects *candidate amount-evidence windows* for VLM reasoning rather than forwarding all edge-retained frames. Specifically, the planner takes

Algorithm 2 Agentic budgeted evidence seeking

Input: item-interaction timeline context C_0, item attributes A, candidate items I, round budget R, per-round window budget B
Output: grounded inventory updates U, unresolved items I_{unres}
Definitions: C: timeline context with appended VLM captions; G_i: grounded amount readings; I_r: unresolved items; $\bar{C}_r$: context filtered to I_r; W_r: selected amount-evidence windows; D_r: VLM captions; ΔG_r: newly grounded readings

$C \leftarrow C_0$; $G_i \leftarrow \emptyset$ for each $i \in I$
for $r = 1$ to R **do**
  $I_r \leftarrow \{i \in I : \neg \textsc{Resolved}(i, G_i)\}$
  if $I_r = \emptyset$ **then**
    break
  end if
  $\bar{C}_r \leftarrow \textsc{FilterContext}(C, I_r)$ ▷ unresolved items only
  $W_r^{\text{stage}} \leftarrow \textsc{StageSample}(\bar{C}_r, A, \{G_i : i \in I_r\})$
  $W_r^{\text{xfer}} \leftarrow \textsc{TransferInspect}(\bar{C}_r, A, \{G_i : i \in I_r\})$
  $W_r \leftarrow \textsc{SelectWindows}(W_r^{\text{stage}} \cup W_r^{\text{xfer}}, B)$
  $(D_r, \Delta G_r) \leftarrow \textsc{VLMReasoner}(W_r)$
  $C \leftarrow C \oplus D_r$ ▷ append captions to context
  for all $(i, \delta_i) \in \Delta G_r$: $G_i \leftarrow G_i \cup \delta_i$ ▷ fill missing readings
end for
$U \leftarrow \{\textsc{Update}(i, G_i) : i \in I \wedge \textsc{Resolved}(i, G_i)\}$
$I_{\text{unres}} \leftarrow \{i \in I : \neg \textsc{Resolved}(i, G_i)\}$
return U, I_{unres}

as input the item-interaction timeline context and the list of candidate inventory items extracted from the edge output. It outputs a bounded set of amount-evidence windows that can ground an amount reading. It uses an agentic LLM as an evidence-seeking planner: it reasons over the temporal structure of item use, the item attributes, and the current evidence needs to decide which short video spans should be inspected by the VLM.

InvAgent define two complementary window types for Agentic LLM Planner to extract from this context. *(1) Stage-sampling windows* provide sparse coverage of an item's interaction timeline by selecting representative frames at key stages, i.e., the sole frame of the item-match peak confidence score timestamp. These windows give the VLM efficient access to candidate pre-use and post-use package states. *(2) Transfer-inspection windows* provide denser evidence by selecting short 3–10 s intervals around likely access, dispensing, transfer, or put-back moments, where occlusion and viewpoint changes make any single frame unreliable. The planner also uses item attributes to adapt its search strategy. In particular, the package transparency tag changes which moments are most valuable: transparent packages often reveal starting or remaining amount in first-seen or final package views, whereas opaque packages require opening, dispensing, or transfer views where the contents are briefly exposed. A selected window may name multiple target item IDs when the same frames cover multiple monitored items. Together, these two window families form the planner's action space in the agentic loop summarized in Alg. 2.

3.3 VLM Reasoner and Agentic Replanning

InvAgent implements the cloud loop as an agentic, budgeted evidence search, summarized in Alg. 2. The algorithm uses the stage-sampling and transfer-inspection window types defined above, while the VLM outputs are detailed below. In Alg. 2, FILTERCONTEXT limits planning to unresolved items, SELECTWINDOWS spends the per-round window budget, and $\oplus$ appends VLM captions to the timeline context.

Grounded amount reading. Selected intervals are sent to the VLM Reasoner in temporal order. For each interval, it produces a

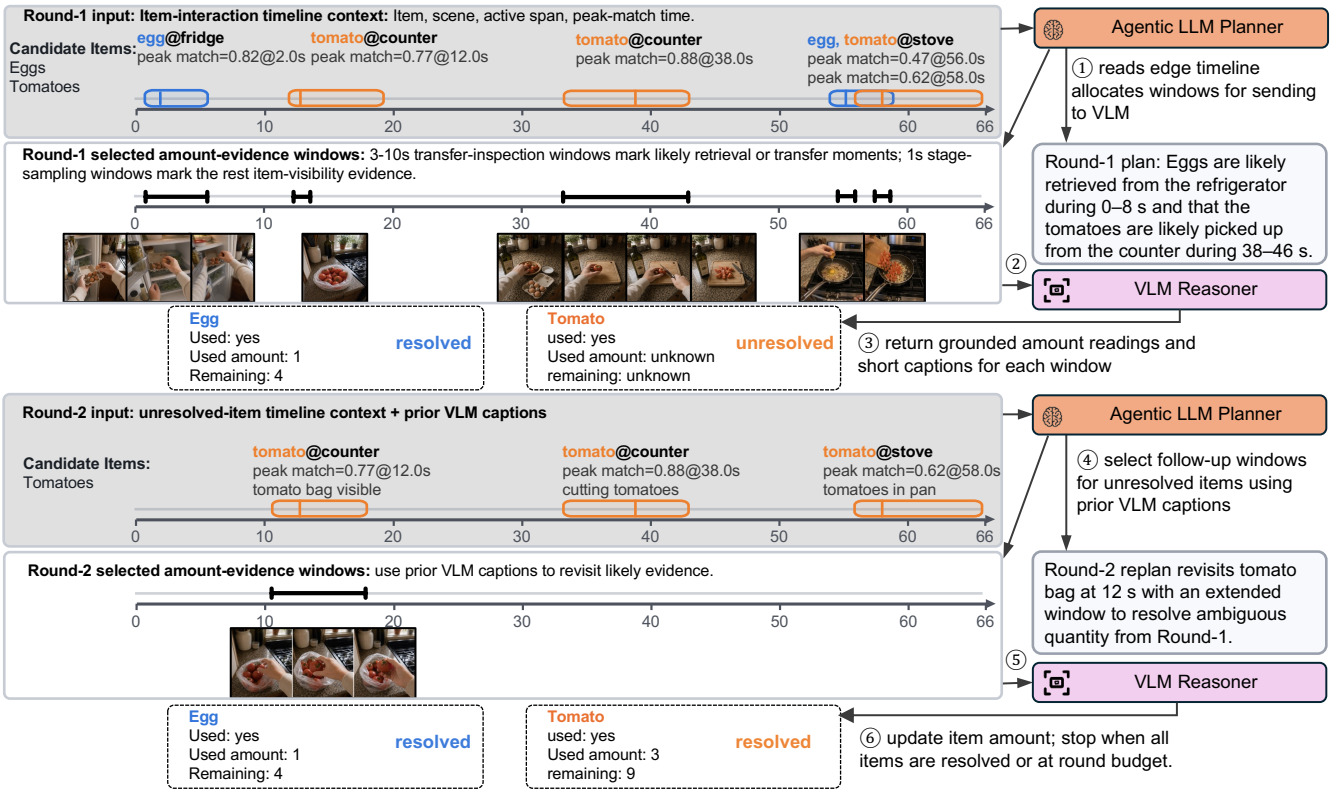


Figure 3: An example of Agentic LLM Planner loop for amount evidence selection. The first Agentic LLM Planner round resolves eggs but leaves tomatoes unresolved because the tomato-bag view is too brief to capture post-use quantity; the next round revisits that bag view with a longer window.

short visual caption and grounded amount fields for each target item: whether the item was used, visible starting amount, visible transferred amount, and visible remaining amount. It also judges whether the selected windows provide sufficient evidence to resolve each item. An item is marked as *resolved* if the remaining amount is directly observed, or if it can be inferred from grounded starting and used amounts; otherwise, it remains *unresolved*. The VLM Reasoner also tags each selected interval as pre-use, during-use, or post-use evidence, allowing the next re-planning round to target only the missing amount evidence.

Agentic replanning. *InvAgent* treats unresolved amount readings as a bounded recovery problem. Because the first planning round is intentionally selective, it may choose windows that verify item use but miss the package-state evidence required for remaining-amount estimation. In Fig. 3, the first VLM call includes only a brief tomato-bag view and later cutting-board and pan views. These windows confirm that tomatoes were used, but they do not show the bag's remaining quantity after tomatoes are taken. In the same round, eggs are resolved because the selected evidence supports an amount update, while tomatoes remain unresolved because the necessary amount fields are missing.

Subsequent rounds filter the planner input to unresolved items and append prior VLM captions to the retained item-interaction timeline context. This keeps resolved items out of the planner prompt and prevents redundant VLM reasoning over previously resolved evidence. In Fig. 3, the second round keeps only tomatoes and revisits the earlier tomato-bag interaction with a longer window that captures the after-taking package state. New VLM outputs fill only missing readings while preserving previously grounded readings; the loop stops when all candidate items are resolved or the round budget is exhausted. This plan–observe–replan design improves evidence coverage while limiting additional cloud cost by replanning only for unresolved items.

Budget-aware reasoning. Deployments may impose a monthly cloud-token budget for recurring inventory monitoring. This creates an allocation challenge across sessions: when the system cannot afford grounded VLM updates for every localized item every day, it must decide which items should receive VLM reasoning now and which can be safely deferred.

InvAgent addresses this with a budget-aware item selection layer before VLM reasoning. The layer uses a depletion-aware item-skipping policy that prioritizes localized candidate items whose inventory state is most uncertain or closest to depletion, while deferring lower-priority items whose projected remaining amount is still sufficient. For each item, *InvAgent* maintains a remaining-amount estimate, initialized from package capacity and overwritten whenever the VLM Reasoner produces a grounded reading. Between grounded readings, this estimate is reduced by a projected per-session usage amount. An item is skipped while its projected remaining amount stays above a configurable low-stock threshold,

and is re-enabled once the projection crosses the threshold or after a configurable number of localized-use sessions without a grounded reading. Skipped items retain their previous or projected inventory state until a later session. This policy provides an explicit monthly cost–accuracy trade-off by focusing VLM reasoning on items where updated quantity information is most valuable.

4 Dataset and Benchmark

The dataset used for our evaluation is collected from 10 participants across 7 households. We define a session as one kitchen activity recording episode with one corresponding pre-session and post-session inventory annotation interval. We focus on daily kitchen activity sessions because they naturally involve frequent inventory-related activities, including retrieving, consuming, and returning inventory items to storage locations. Participants record first-person video using either **Meta Ray-Ban Smart Glasses** [20] or our own smartglass prototype with a head-mounted **Raspberry Pi Camera Module v3** [36], depending on hardware availability. The source recordings are captured at 30 fps by default. Raspberry Pi sessions use 1920×1080 video, while Meta Ray-Ban sessions are stored as portrait videos with varying encoded resolutions, all at least 1376×1840.

4.1 Collection Protocol

Item initialization. Before recording any session, each food item is initialized in the participant's inventory list using its grocery receipt. For each item, we record its name, its *original package size* (in grams or count), a product image of the package from the retailer's product page, and the package-transparency tag used by the planner. These attributes are populated once when the item enters the household.

Data collection scenarios. Two complementary cohorts are collected: *(1) In-the-wild sessions*, which capture participants performing natural kitchen activities in their homes and serve as the primary realistic-deployment setting for system and cost evaluation; (2) *Controlled-use sessions*, which broaden item-category coverage and support targeted VLM amount-reading diagnostics.

(1) In-the-wild kitchen activity sessions. Participants wear the recording device during their kitchen activity at home, without any instruction on what food to use or how to prepare it. At the beginning and end of each recording, the participant places each to-be-used or just-used item on a kitchen scale and records the weight, producing the per-item ground-truth amounts. The participants are explicitly *not* asked to keep the item in view or to avoid natural recording noise; hand obstruction, head turning, off-camera handling, and the usual artifacts of egocentric recording are kept in the data so that the cohort reflects the realistic deployment scenario.

(2) Controlled-use sessions. Participants follow a scripted protocol that uses pre-selected items: each item is removed from storage, a known amount is consumed, and the item is returned. This cohort exists to deliberately exercise items under-represented in free-form kitchen activity, and is used for targeted amount-reading diagnostics rather than deployment-cost evaluation.

Per-session annotation. For every item used in a session, the participant records the measurements needed to reconstruct its session-level inventory change: a pre-session amount, measured by weight for *bulk* items, where the remaining amount is continuous (e.g., fluids, granular foods, and cuttable solids), and by count for *discrete* items, where inventory changes in countable units (e.g., eggs or yogurt cups); the ending amount in the same unit; and whether the item was depleted and discarded during the session. Before evaluation, we redact any video frames in which the scale display reveals the measured amount of a tracked inventory item, so a VLM cannot read the amount directly from the recording.

4.2 Dataset summary

Item Variety. Our dataset contains 254 item instances. We categorize these items based on their physical form and whether their amount is measured by weight or count. The categories include fluids such as olive oil, milk, and soy sauce; granular items such as rice, pasta, and blueberries; solids such as garlic, steak, and tofu; and discrete count-measured items such as eggs, yogurt cups, and bagels. Overall, it includes 81 discrete, 78 solid, 61 granular, and 34 fluid items. Only discrete items are measured by count, while the other categories are measured by weight. In addition, each item is assigned a package-transparency tag (Sec. 3.1). This tag helps choose amount-evidence windows: transparent packages support retrieval or put-back fill-level views, whereas opaque packages require access or use windows that show the open package interior or post-use contents.

Dataset coverage. Table 5 reports in-the-wild dataset coverage across anonymized households. This in-the-wild cohort contains 38.97 hours of video and 604 usage events. A subset of the in-the-wild kitchen activity sessions is used for system and cost evaluation, while all annotated sessions are used by default for inventory monitoring accuracy evaluation.

Longitudinal coverage. A key property of this dataset is its longitudinal coverage. The same item often persists across multiple sessions before being exhausted, allowing us to evaluate *how a system tracks a package over its depletion trajectory* rather than only its performance on isolated sessions. Across the annotated in-the-wild inventory lists, 245 item instances appear in at least one session, 67 in at least three sessions, 29 in at least five sessions, 12 in at least ten sessions, and 4 in at least fifteen sessions. The longest-running items, such as eggs, cheddar cheese, cereal, oats, and milk, are observed across 13–18 sessions each. This long-tail coverage makes the benchmark sensitive to amount updates across repeated uses, rather than only one-off item detection.

5 Implementation and Evaluation

5.1 System Implementation

We implement *InvAgent* as a hybrid edge–cloud system. The Edge Context Builder is designed to run on a household edge endpoint. For edge-overhead evaluation (Sec. 5.6), we prototype this deployment target on an NVIDIA Jetson AGX Orin [27]. For the main accuracy and cloud-cost experiments, we execute the same Edge Context Builder on an RTX 6000 Ada server [28] to accelerate evaluation. Because both platforms run the same edge algorithm, this choice affects only preprocessing runtime, not cloud-token accounting or inventory metrics. The Edge Context Builder samples video at 1 fps by default. Guided by the Agentic LLM Planner's selected time windows, the edge device uploads the corresponding frames to the cloud for VLM reasoning. On the cloud side, the Agentic LLM

Table 1: Paid-tier model pricing.

Model	Role	p_{in} (\$/M)	p_{out} (\$/M)
Gemini-3.1 Pro (input $\leq$ 200K)	VLM Reasoner	2.00	12.00
Gemini-3.1 Pro (input $>$ 200K)	VLM Reasoner	4.00	18.00
Gemini-3 Flash	VLM Reasoner	0.50	3.00
GPT-5.4	LLM Planner	2.50	15.00

Planner uses GPT-5.4 with `reasoning.effort` set to `medium`, and the VLM Reasoner uses Gemini-3.1 Pro over selected video frames with `thinking_level` set to `high`. For both models, we record input tokens, visible output tokens, and internal reasoning or thinking tokens. Cloud-cost calculations count internal reasoning/thinking tokens as output tokens, using May 2026 paid-tier rates. Unless otherwise specified, *InvAgent* uses a three-round plan–observe cap per session to bound cloud cost and latency. Budget-aware reasoning is disabled in the main accuracy and cloud-cost experiments: *InvAgent* attempts grounded VLM updates for every localized candidate item. We evaluate budget-aware reasoning separately later in this section. To enable fair comparison, we also implement all baselines using the same default settings. The detailed implementation of each baseline is discussed in the following subsections.

5.2 Evaluation Metrics

We evaluate whether *InvAgent* can support resource-efficient daily inventory monitoring through agentic edge-cloud orchestration from three perspectives, including *(1) cloud-based VLM cost*, *(2) inventory monitoring accuracy*, and *(3) edge computation overhead*.

Cloud-based VLM cost. We estimate cloud cost based on the paid-tier pricing of each model used in our pipeline [12, 30]. For each cloud call c, the input tokens $T_{\text{in}}(c)$ and output tokens $T_{\text{out}}(c)$ are charged at the corresponding per-million-token rates $p_{\text{in}}(m_c)$ and $p_{\text{out}}(m_c)$, where m_c denotes the model serving call c. T_{out} includes both visible response tokens and any thinking or reasoning tokens that are charged at the output-token rate by the provider's API convention [12, 30]. For image inputs, we convert each frame to 280 tokens following Gemini 3 media-resolution accounting [11]. Table 1 summarizes the paid-tier rates used in all cost calculations [12, 21]. For each evaluation cell, the total cloud cost sums all Agentic LLM Planner calls and VLM Reasoner calls. Gemini-3.1 Pro applies the 200K-token tier split per call: at 280 tokens per frame, a single call crosses the threshold once it carries more than ≈ 714 frames, i.e. $\sim$ 12 minutes of 1 fps video. No *InvAgent* call crosses the threshold: the maximum per-call input is $\approx$ 31K tokens.

Two key evaluation metrics are used:

(1) Billable tokens T_{all}: To provide a provider-agnostic comparison, we report billable tokens as: $T_{\text{all}} = T_{\text{in}} + \alpha T_{\text{out}}$ We set $\alpha = 6$, matching the output-to-input price ratio of all three models on their primary pricing tier.

(2) Monthly cloud cost C_{month}: To make costs comparable under a recurring household deployment scenario, we estimate monthly cost by assuming one hour of daily logging based on the 2024 American Time Use Survey [47]. $C_{\text{month}} = \frac{30 C_{\text{cell}}}{H}$, where C_{cell} is the total cloud cost for an experimental cell, H is the number of recorded video hours in that cell, and we assume 30 days per month.

Inventory monitoring accuracy. We evaluate whether the system produces valid inventory updates for used items using the following metrics. Let G denote the set of ground-truth item-use events in a session, and let $\mathcal{P}$ denote the set of system-predicted used-item records. A prediction is considered matched if it can be associated with a ground-truth item-use event. Let $M \subseteq G$ be the matched ground-truth events, $U = G \setminus M$ be missed events, and $H \subseteq \mathcal{P}$ be hallucinated predictions.

(1) Amount-update coverage rate (%): Among the matched events, let $M_{\text{amt}} \subseteq M$ denote the subset for which the system returns a numeric remaining-amount estimate at the end of a session. We define amount-update coverage as $|M_{\text{amt}}|/|G|$. Thus, a detected item-use event is credited only if it also produces a numeric remaining-amount estimate; otherwise, it does not update the inventory state. Items skipped by the Agentic LLM Planner, missed by the Edge Context Builder, or returned as unresolved are counted as misses.

(2) Hallucinated-update rate (%): A hallucinated update is a predicted used-item record that cannot be matched to any ground-truth item-use event in the session. We define the hallucinated-update rate as $|H|/(|G|+|H|)$. This workload-normalized diagnostic measures how much the inventory-update stream is polluted by extra predicted updates when meaningful true negatives are not available.

(3) Capacity-normalized percentage error (CNPE, %): CNPE evaluates how accurately *InvAgent* estimates the numeric remaining amount of an item. For each matched event $g \in M_{\text{amt}}$, let $\hat{r}_g$ and r_g denote the predicted and ground-truth session-end remaining amounts, and let C_g denote the package capacity. We compute the normalized per-event error $\text{CNPE}_g = |\hat{r}_g - r_g|/C_g$. CNPE is computed in percentage as the average over all events in M_{amt}. Because CNPE conditions on a numeric prediction and does not penalize missed or hallucinated updates, we use it as a diagnostic metric for VLM-based remaining-amount estimation.

(4) Amount-aware inventory F1 (%): We evaluate the end-to-end inventory update quality with an amount-aware F1 score. For each $g \in G$, let $e_g = |\hat{r}_g - r_g|/C_g$ be its capacity-normalized remaining-amount error if matched to a numeric prediction, and set $q_g = 1 - \min(e_g, 1)$. Unmatched events or matched events without a numeric amount estimate have $q_g = 0$. With $Q = \sum_{g \in G} q_g$, the recall and precision are $R_{\text{amt}} = Q/|G|$ and $P_{\text{amt}} = Q/|\mathcal{P}|$, respectively. Amount-aware inventory F1, $F1_{\text{inv}}$, is the harmonic mean of R_{amt} and P_{amt}, reported as a percentage. When $\mathcal{P}$ is empty, we set P_{amt} and $F1_{\text{inv}}$ to 0. This metric penalizes missed events through recall, hallucinated predictions through precision, and amount errors through q_g.

Overall, better inventory monitoring accuracy corresponds to higher amount-update coverage, lower hallucinated-update rate, lower CNPE, and higher amount-aware inventory F1.

Edge processing requirements. Finally, since the main accuracy and cloud-cost experiments rely on edge outputs, we measure the Edge Context Builder's local processing requirements on a household edge endpoint. For each edge stage, we report *(1) processing time as seconds per sampled frame*, and *(2) peak GPU memory*. For each model stack, we report *amortized edge latency*: the sum of each model's measured latency weighted by how often that model is invoked by the gated pipeline, divided by the number of sampled frames. We also report edge processing time normalized by recorded-video duration with the default 1 fps sampling.

Table 2: Baseline accuracy and projected monthly cost. F, C, and H denote amount-aware inventory F1 (%), amount-update coverage (%), and hallucinated-update rate (%), respectively. T_{in} and T_{out} denote monthly input and output tokens in millions. Bill. denotes provider-agnostic billable tokens in millions. Cost denotes projected monthly cost in USD.

				Agentic LLM Planner		VLM Reasoner			
Method	F	C	H	T_{in} (M)	T_{out} (M)	T_{in} (M)	T_{out} (M)	Bill. (M)	Cost (USD/mo.)
Whole-video VLM	66.6	90.9	5.5	0.00	0.00	37.71	2.39	52.05	135.40
No-planner (all segments)	67.7	89.9	8.2	0.00	0.00	15.69	1.19	22.81	49.63
No-planner (uniform sampling)	64.2	85.4	9.3	0.00	0.00	3.72	1.20	10.90	16.77
InvAgent	**68.9**	90.2	6.2	0.62	0.79	3.20	1.22	15.88	26.51

5.3 Micro Benchmark and Baseline Comparison

We compare three baselines with *InvAgent* on in-the-wild sessions.
(1) Whole-video VLM. This baseline sends the entire video session to the cloud-based VLM for reasoning, with the video uniformly sampled at 1 FPS.
(2) No-planner (all segments). This baseline uses the Edge Context Builder in *InvAgent* to identify all potentially relevant video segments and sends all selected frames to the cloud-based VLM for reasoning, without invoking the Agentic LLM Planner. Thus, it removes Agentic LLM Planner overhead but may require the VLM to process more frames.
(3) No-planner (uniform sampling). This baseline also uses the Edge Context Builder in *InvAgent* to identify localized item-interaction segments. Rather than using the Agentic LLM Planner to adaptively select amount-evidence windows, it forms the chronological union of frames selected from the Edge Context Builder output and uniformly subsamples that pool to a 100-frame cap, matching *InvAgent*'s frame-budget ceiling. This baseline evaluates whether the Agentic LLM Planner provides benefits beyond simply imposing a fixed frame budget on the Edge Context Builder output.

Table 2 summarizes our micro benchmark results. *Compared with whole-video VLM, InvAgent reduces total cloud input tokens by* 9.9× *(37.71M to 3.82M) and projected monthly cost by* 5.1× *while achieving the best amount-aware inventory F1 (*68.9%*).* This token-volume gap is compounded by per-call tier pricing: because whole-video issues one call per session and 41% of wild sessions exceed 12 minutes, 58 of 140 whole-video calls (and 24 of 140 no-planner all-segment calls) bill at the higher 200K-token tier, whereas every *InvAgent* call stays on the cheaper tier. Compared with no-planner all-segments, *InvAgent* reduces VLM Reasoner input tokens by 4.9× (15.69M to 3.20M) and projected monthly cost by 1.9×. Compared with no-planner uniform sampling, *InvAgent* spends more cloud budget due to Agentic LLM Planner overhead but improves amount-aware inventory F1 from 64.2% to 68.9% and reduces hallucinated updates from 9.3% to 6.2%. Together, these baselines show that edge-cloud orchestration is necessary. The Agentic LLM Planner improves the cost–accuracy tradeoff by selecting and revisiting amount-evidence windows under a bounded budget.

5.3.1 VLM amount-reading breakdowns. The primary results show that the remaining accuracy gap is driven less by missing updates or hallucinated items than by amount reading. On the in-the-wild cohort, *InvAgent* reaches 90.2% amount-update coverage and a 6.2% hallucinated-update rate, yet its amount-aware inventory F1 is still only 68.9%. This gap points to the VLM Reasoner's numeric amount-reading bottleneck. We therefore isolate matched numeric updates and ask where the VLM amount-reading error is largest.

Table 3: Matched-only VLM amount error by item type.

	Shape category				Package tag	
Metric	Fluid	Granular	Solid	Discrete	Transparent	Opaque
n	142	146	119	166	356	217
CNPE_{m} (%)	27.1	23.1	**19.0**	24.1	**22.5**	25.2

Matched amount reading. Table 3 isolates amount-reading error after *InvAgent* has already emitted a numeric update. It reports matched-only CNPE (CNPE_{m}), the remaining-amount error over matched events with numeric predictions, which should be read as a VLM capability breakdown rather than an end-to-end inventory metric. This diagnostic pools matched numeric updates from both in-the-wild and controlled-use sessions, so that under-represented item forms and package types contribute to the VLM capability analysis. The results show that, given amount evidence, the VLM Reasoner estimates remaining amount coarsely across item and package types, but fluids and opaque packages remain harder.

Solid items have the lowest matched CNPE (19.0%), because the remaining amount is often visible as a coherent object. Fluids are hardest (27.1%): container geometry, tilt, and hidden volume can remain ambiguous even in useful fill-level views. Transparency also helps, but only modestly: transparent packages are 2.7 CNPE points better than opaque packages (22.5% vs. 25.2%).

Overall, VLM amount reading is useful for coarse, evidence-grounded inventory state, but it remains visually limited. *InvAgent*'s role is to expose the right evidence cheaply; it does not remove the underlying amount-reading bottleneck.

5.4 Ablation Study

We evaluate the contribution of each design component by removing *(1)* scene tags from Pre-Frame Context Builder and *(2)* package-transparency hints from Item Attributes in Sec. 3.1. To evaluate the contribution of the agentic replanning part, we also vary the Agentic LLM Planner round budget and compare one-, two-, three-, and four-round configurations. Table 4 summarizes the ablation study of *InvAgent*.

Comprehensive edge-generated context, including scene tags and package-transparency hints (Sec 3.1), is important for both accuracy and cost. Removing scene tags reduces amount-aware inventory F1 by 1.9% relative to *InvAgent* and increases the hallucinated-update rate from 6.2% to 7.1%. Removing package-transparency

Table 4: Planner-design ablations. F, C, and H are percentages following Table 2.

Setting	F	C	H	Cost (USD/mo.)
1 round	62.2	79.9	5.9	23.10
2 rounds	67.9	88.7	6.1	25.76
3 rounds (*InvAgent*)	68.9	90.2	6.2	26.51
4 rounds	69.1	90.6	6.2	26.67
No scene tags	67.0	89.6	7.1	28.21
No transparency hint	66.5	88.4	6.2	27.33

hints reduces amount-aware inventory F1 by 2.4 percentage points and lowers update coverage from 90.2% to 88.4%. Both cues also reduce projected monthly cost: the full edge context costs $26.51 per month, compared with $28.21 without scene tags and $27.33 without package-transparency hints. These cues help the Agentic LLM Planner avoid reasoning over item-interaction context that are unlikely to support inventory updates.

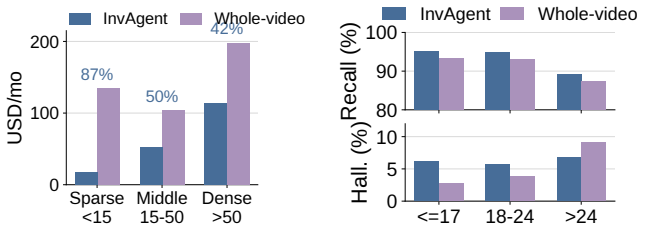
The Agentic LLM Planner (Sec. 3.2 and Sec. 3.3) improves inventory monitoring by agentic multi-round amount evidence detection. A single-round Agentic LLM Planner achieves only 62.2% amount-aware inventory F1 at a projected monthly cloud cost of $23.10, while two rounds substantially improve F1 to 67.9% with a modest cost increase to $25.76. The default three-round configuration further improves F1 to 68.9% at $26.51, capturing most of the replanning benefit before the curve flattens (four rounds: 69.1% F1 and 90.6% update coverage). Notably, additional rounds introduce diminishing cost overhead because replanning is increasingly targeted: the second round adds $2.66 per month, whereas the third and fourth rounds add only $0.75 and $0.16, respectively. This shows that once the Agentic LLM Planner resolves the major missing-evidence cases, later rounds only query a small set of harder residual cases. The extra agentic replanning also remains selective overall: the three-round configuration sends 10,329 frames to the VLM, compared with 55,312 frames for *no-planner (all-segments)* baseline, and uses 3.20M VLM input tokens compared with 15.69M in Table 2. *InvAgent* is three rounds by default, which captures most of the useful replanning benefit before the accuracy–cost curve flattens.

Budget-aware reasoning (Sec. 3.3) can further reduce the end-to-end cost. To demonstrate the cost–recall tradeoff introduced by budget-aware reasoning, we use the data from Household A for evaluation, which spans 47 days and includes 19 items observed from initial use through depletion with all usage events captured. It skips 157 of the 379 monitored item-sessions (41.4 %), reducing VLM Reasoner 21 % cost for Household A. This result shows that item skipping gives *InvAgent* a direct way to reduce cloud cost under a token budget.

5.5 Generalization Evaluation

Next, we evaluate whether *InvAgent*'s cost–accuracy trade-off generalizes across recording conditions and deployment choices, including item-use event density, household and subject variation, monitoring-list size, and VLM reasoner choice.

Item-use event density generalization. Monthly prices differ across users partly because kitchen recordings vary in densities of inventory item-use events. Sparse recordings contain long intervals without retrieval, opening, dispensing, or put-back actions, whereas dense recordings concentrate many such interactions into a short



(a) Savings are largest in sparse sessions and taper as item-use event density increases.

(b) Recall and hallucinated-update rate across different active monitoring-list size.

Figure 4: Generalization trends.

duration. Because *InvAgent* localizes candidate item interactions before cloud reasoning, its cloud cost scales with the total duration of localized amount-evidence windows rather than with recording duration. Fig. 4a groups sessions by ground-truth item-use event density. For each bin, we compute *InvAgent*'s projected monthly cost; labels report the cost reduction relative to whole-video VLM inference. Sparse sessions, defined as fewer than 15 item-use events per hour, account for 27.0 of the 39.0 in-the-wild recording hours but contain only 200 item-use events. In this setting, *InvAgent* costs only $17.35 per projected month, 87% lower than whole-video inference. For dense sessions, the savings decrease to 42% because much of the recording contains interactions requires VLM reasoning. *Item-use event density is an important generalization axis: InvAgent saves the most when recordings are dominated by non-interaction time, and it naturally spends more when the user performs many inventory interactions.* This result also contextualizes the per-household comparison below: *different household costs may reflect different interaction densities rather than a failure to generalize across users.*

Household and subject generalization. Table 5 tests whether the cost–accuracy trade-off holds across different users and homes. *Across all seven households, InvAgent preserves the pooled trade-off: it cuts projected monthly cost by* 68–92% *and improves amount-aware inventory F1 over whole-video VLM in five households.* In the two smallest cohorts, E and F, with only 1.4 and 1.2 recorded hours, respectively, whole-video inference achieves higher F1. Even in these cases, *InvAgent* can still maintain the F1 while reducing projected monthly cost by 76% and 68%. The variation in household-level cost and F1 mainly reflects differences in item-use event density and the types of monitored inventory items, rather than a systematic failure to generalize across users.

Monitoring-list size generalization. Monitoring more active inventory items stresses *InvAgent* in a way that is not captured by item-use event density alone: more items can compete for the same bounded evidence-seeking budget. We therefore bucket sessions by active monitoring-list size to test where the current design begins to lose recall. In *InvAgent*, this list is used by the Edge Context Builder for product-instance retrieval; in the whole-video baseline, the same item names are included in the VLM prompt.

Fig. 4b shows the results. *InvAgent* maintains high recall for small and medium lists (95.1% and 95.0%), but drops to 89.1% once the active monitoring list exceeds 24 items. The whole-video baseline follows a similar recall drop, confirming that larger candidate

Table 5: Per-household core-method breakdown. Household headers report video hours and event counts; the full dataset contains 38.97 hours and 604 events. F, C, and H are percentages following Table 2. $ denote projected cost (USD/mo.).

	A (15.90 h, 372 events)				**B** (4.48 h, 38 events)				**C** (8.12 h, 34 events)				**D** (3.64 h, 36 events)				**E** (1.43 h, 36 events)				**F** (1.21 h, 25 events)				**G** (4.18 h, 63 events)			
Method	F	C	H	$	F	C	H	$	F	C	H	$	F	C	H	$	F	C	H	$	F	C	H	$	F	C	H	$
Whole-video VLM	67.2	89.8	6.3	130.03	61.3	97.4	5.0	129.22	63.1	91.2	2.9	147.08	57.7	100.0	0.0	133.33	82.9	100.0	2.7	155.99	65.0	80.0	3.9	136.63	64.1	87.3	7.4	134.46
No-planner all	66.3	90.6	7.3	50.21	71.7	94.7	5.0	42.13	77.6	100.0	19.0	43.47	56.2	75.0	5.3	64.24	70.7	91.7	7.7	36.83	60.9	68.0	3.9	62.34	74.7	93.7	12.5	55.56
No-planner uniform	63.6	86.5	8.4	22.65	68.6	89.5	7.3	8.70	70.1	91.2	22.7	8.03	46.9	63.9	2.7	12.32	70.2	94.4	7.7	23.62	61.5	72.0	7.4	28.63	68.1	85.7	12.5	18.23
InvAgent	68.1	89.8	5.4	36.16	67.2	92.1	5.0	14.95	74.6	97.1	17.1	11.84	61.3	86.1	2.7	24.69	75.1	91.7	5.3	38.12	61.8	80.0	7.4	43.60	74.2	93.7	7.4	23.44

Table 6: Gemini Pro/Flash sensitivity. F, C, and H are percentages; Cost denotes projected cost (USD/mo.), following Table 2.

Configuration	F	C	H	T_{in}	T_{out}	Bill.	Cost
Whole-video VLM Gemini-3.1 Pro	66.6	90.9	5.5	37.71	2.39	52.05	135.40
Whole-video VLM Gemini-3 Flash	67.0	96.7	8.9	41.37	1.34	49.42	19.02
InvAgent **GPT-5.4 + Gemini-3.1 Pro**	**68.9**	90.2	6.2	3.82	2.01	15.88	26.51
InvAgent **Gemini-3 Flash**	65.2	91.2	9.9	3.79	2.32	17.71	6.82

inventories make grounding harder for both approaches. In comparison, *InvAgent* keeps the hallucinated-update rate stable (6.2% to 6.8%) as the monitoring-list size increases, whereas the *whole-video VLM* baseline shows a substantial increase in hallucinated-update rate (2.7% to 9.1%). This is because the Edge Context Builder of *InvAgent* identifies the candidate items from the list to the cloud VLM reasoner while *whole-video VLM* baseline takes the full item list as the input.

Model Generalization. Table 6 shows whether *InvAgent*'s benefit persists with a cheaper VLM Reasoner by replacing the default Gemini 3.1 Pro with Gemini 3 Flash. Gemini 3 Flash lowers cost but weakens fine-grained amount reading: *InvAgent* drops 3.7% F1 relative to Gemini 3.1 Pro and trails whole-video Flash by 1.8%. Nevertheless, *with Gemini 3 Flash, InvAgent still reduces projected monthly cloud cost by* 2.8× *compared with the* whole-video VLM *baseline ($19.02 to $6.82).* We did not use GPT-5.4 as the VLM reasoner as we found that our Azure deployment caps requests at 50 images, incompatible with the whole-video baseline.

5.6 Edge Runtime and Deployment Feasibility

The accuracy and cloud-cost results above use our default CV models for Edge Context Builder. We next ask *(1)* whether the models can run on an in-home GPU endpoint (i.e., Jetson AGX Orin), and *(2)* how edge model choices change the end-to-end cost–recall tradeoff. We profile hand-object interaction detector (i.e., Hands23) on sampled frames, object identification model (i.e., DINOv3-7B or DINOv3-L) on the hand-object interaction crops, and the scene recognition model (i.e., OWLv2 or YOLOE) on frames that pass item matching. On Jetson, Hands23, DINOv3 and YOLOE run in PyTorch on CUDA, while OWLv2 is exported to ONNX [29] and compiled with TensorRT using the same 960 px input, query set, and post-processing as the PyTorch model.

Table 7: Per-model Jetson profiling for the Edge Context Builder. DINOv3 latency is measured per HOI crop; Hands23, OWLv2, and YOLOE latency are measured per sampled frame.

Model	Peak GPU memory (GB)	Latency
Hands23	0.84	0.337 s/fr
DINOv3-7B	12.61	0.318 s/crop
OWLv2	0.285	0.400 s/fr
DINOv3-L (alt.)	0.63	0.063 s/crop
YOLOE scene (alt.)	0.072	0.056 s/fr

Edge model GPU memory and latency. Table 7 reports Jetson AGX Orin latency and peak GPU memory for each model. For the default stack, the summed peak memory of the resident models is 13.74 GB. Over the in-the-wild evaluation set (141,028 sampled frames, ≈ 39.2 h at 1 fps), Hands23 flags 110,109 item-interaction frames, producing 211,303 DINOv3 crops; item matching then activates OWLv2 on 58,902 frames, yielding an amortized edge latency of ≈ 0.98 s per sampled frame for the default stack. We next swap the object identification and scene recognition models to lighter alternatives, i.e., DINOv3-L (a 300M-parameter model distilled from DINOv3-7B [43]) and YOLOE. The fully lightweight edge model choices (DINOv3-L + YOLOE) reduces summed peak GPU memory by 9× (13.74 to 1.54 GB) and amortized latency by more than half (0.97 to 0.45 s/frame).

Cost-recall trade-off across different edge model choices. When switching to the lighter alternatives, *InvAgent* preserves similar amount-aware inventory F1 and update coverage, as shown in Tab. 8. The tradeoff shifts cost to the cloud: monthly cloud cost increases by 17% ($30.96 vs. $26.54), and the hallucinated-update rate rises from 6.2% to 7.4% when using the DINOv3-L + YOLOE. Holding OWLv2 fixed, replacing DINOv3-7B with DINOv3-L makes crop encoding 5.0× faster and reduces item-matcher peak memory by about 20×, but raises cloud cost by 16% because the smaller visual embedding is less selective: more spurious item candidates appear in the edge-produced item-interaction timeline context, so the planner forwards more evidence to the VLM Reasoner and spends more VLM tokens. Replacing OWLv2 with YOLOE has a smaller effect. Thus, *InvAgent* can use the default stack when minimizing cloud reasoning is preferred, or a lighter stack when the edge device has a tighter latency or GPU-memory budget.

6 Related Work

InvAgent sits at the intersection of long-video understanding, object-state understanding, and egocentric-video benchmarks, but targets a different systems problem: recurring package-level inventory updates under a priced cloud VLM budget.

Table 8: Edge-model choices trade local memory and latency against cloud cost. Latency is amortized edge latency.

Scene tagger	Item matcher	F	C	H	USD /mo	Mem. (GB)	Amort. lat. (s/fr)
OWLv2	DINOv3-7B	69.0	90.5	6.2	26.54	13.74	0.97
YOLOE	DINOv3-7B	67.8	88.9	6.4	27.09	13.52	0.83
OWLv2	DINOv3-L	70.0	89.7	6.2	30.76	1.76	0.59
YOLOE	DINOv3-L	69.0	89.9	7.4	30.96	1.54	0.45

Long video understanding. Egocentric long video understanding systems exploit hand-object interactions, repeated places, and persistent scene structure to build affordance maps, active-object memories, and scene memories for later retrieval and reasoning [8, 10, 16, 24, 25, 34, 35, 40]. However, they do not support revisiting visual evidence to understand fine-grained object states such as food quantity. Recent agentic video understanding systems address long-context limits by compressing video into memories, retrieving candidate evidence spans, and using LLM/VLM agents to decide what to inspect next [9, 39, 42, 52, 53, 56, 57, 59]. Their optimization target is accuracy under the VLM context-window bottleneck: the agent retrieves or selects evidence so the model need not ingest the entire recording at once. When these systems report efficiency, they usually measure retrieved frames, input-token reduction, or inference time, rather than the priced cloud cost of a recurring deployment. *InvAgent* instead applies this pattern to recurring, resource-constrained inventory monitoring: the Edge Context Builder constructs item-interaction timeline context, and the Agentic LLM Planner reserves cloud VLM reasoning for selected moments that reveal food amount.

Object state-change understanding. Existing benchmarks and methods ask VLMs or video models to identify symbolic states such as open/closed or on/off, infer multi-label food states such as cracked or whisked, predict how several objects' states evolve through procedural actions, or track an object through a visible transformation [44–46, 54]. These tasks demonstrate that object state is temporally grounded and often depends on action history. However, they typically operate on image pairs, narrated or action-segmented procedural videos, or tracked object tubelets in which the relevant object interaction is already in view. Our setting instead starts from long untrimmed household video and an inventory list of known items. The system must retrieve the few visual moments that can update each inventory-list entry, estimate a remaining amount in weight or count, and do so under a cloud VLM token budget.

Egocentric video benchmarks. Egocentric video benchmarks provide the closest dataset context for our task, but their annotation targets differ from ours. They provide action segments, hand-object masks and relations, narrations, recipe steps, ingredients, object motion, VQA labels, and procedure-error labels [2, 4, 5, 13–15, 17, 18, 31–33, 51, 58]. These labels do not evaluate systems' capability to recover the remaining quantity of a food package. Our benchmark therefore targets a different state variable: package-level food amount, measured by weight or count, for known inventory items across natural kitchen activity sessions.

7 Discussion

Amount reading is bounded by visual observability. The planner can request additional windows, but cannot recover amount when no window exposes a readable package state. Opaque or reflective containers, fluids, and near-empty packages remain difficult even after relevant interactions are selected (Table 3). For instance, olive oil and soy sauce are among the highest-error items in our traces: the item is clearly used, but the remaining amount stays hidden inside an opaque container no matter which window is selected.

Low-stock decisions support only coarse reminders. We also tested whether predicted remaining amounts support thresholded low-stock decisions. At a 50% capacity threshold, *InvAgent* reaches 68.5% F1, so this signal is best suited to coarse reminders such as "check this item before shopping." The stricter 25% threshold remains far from automation quality. These results do not support autonomous grocery purchasing, nutrition logging, or precise recipe planning, and they reinforce the amount-reading limits seen in Table 3.

Smartphones are not yet practical edge endpoints. Commodity smartglasses such as Ray-Ban Meta use smartphones as a controller and internet portal [37], making the phone a natural first candidate for the edge role. However, our inventory-interaction discovery stack requires both hand-object contact detection and product-instance retrieval. On a Pixel 9 Pro trace, the phone takes 2.67 s per sampled frame for Hands23 and 9.37 s per interaction crop for DINOv3-L encoding. Alternatives such as MediaPipe Hands and YOLO-style detectors can reduce phone compute [38, 55], but lower contact or retrieval accuracy would either miss item-use evidence or force the system to forward more candidates to the cloud VLM, increasing token cost. Future work should explore this tradeoff among compute, recall, and cloud costs for deploying the video context builder on smartphones. At the same time, emerging GPU-class AI PCs such as NVIDIA RTX Spark suggest a household edge point between smartphones and cloud servers, making *InvAgent*-style local context construction increasingly practical in homes [22, 26].

8 Conclusion

InvAgent demonstrates that long-horizon inventory tracking can avoid whole-video cloud VLM reasoning. By constructing item-interaction timeline context on the edge and using an agentic LLM to select VLM reasoning windows, *InvAgent* preserves inventory-update quality while reducing projected monthly cloud cost by 5.1×. These results support agentic edge–cloud orchestration as a practical path for wearable memory systems that need fine-grained state updates without streaming every frame to cloud VLMs.

Ethics Statement

This study involving human participants was reviewed and approved by an institutional review board. Participants provided informed consent before recording. All participant and household identifiers are anonymized.

References

[1] Centers for Disease Control and Prevention. 2025. Facts About Food Poisoning. https://www.cdc.gov/food-safety/data-research/facts-stats/index.html. Content current as of November 24, 2025; accessed May 20, 2026.

[2] Yi Chen, Yuying Ge, Yixiao Ge, Mingyu Ding, Bohao Li, Rui Wang, Ruifeng Xu, Ying Shan, and Xihui Liu. 2026. Egoplan-bench: Benchmarking multimodal large

language models for human-level planning. *International Journal of Computer Vision* 134, 3 (2026), 118.

[3] Tianyi Cheng, Dandan Shan, Ayda Sultan Hassen, Richard E. L. Higgins, and David F. Fouhey. 2023. Towards A Richer 2D Understanding of Hands at Scale. *Advances in Neural Information Processing Systems* 36 (2023). https://proceedings.neurips.cc/paper_files/paper/2023/hash/612a7948f3294a02a63d970566ca8536-Abstract-Conference.html

[4] Dima Damen, Hazel Doughty, Giovanni Maria Farinella, Antonino Furnari, Evangelos Kazakos, Jian Ma, Davide Moltisanti, Jonathan Munro, Toby Perrett, Will Price, and Michael Wray. 2022. Rescaling Egocentric Vision: Collection, Pipeline and Challenges for EPIC-KITCHENS-100. 130, 1 (2022), 33–55. doi:10.1007/s11263-021-01531-2

[5] Ahmad Darkhalil, Dandan Shan, Bin Zhu, Jian Ma, Amlan Kar, Richard Higgins, Sanja Fidler, David Fouhey, and Dima Damen. 2022. *EPIC-KITCHENS VISOR Benchmark: VIdeo Segmentations and Object Relations.* arXiv:2209.13064 [cs] doi:10.48550/arXiv.2209.13064

[6] Ellen W. Evans and Elizabeth C. Redmond. 2014. Behavioral Risk Factors Associated with Listeriosis in the Home: A Review of Consumer Food Safety Studies. *Journal of Food Protection* 77, 3 (2014), 510–521. doi:10.4315/0362-028X.JFP-13-238

[7] Ellen W. Evans and Elizabeth C. Redmond. 2015. Analysis of Older Adults' Domestic Kitchen Storage Practices in the United Kingdom: Identification of Risk Factors Associated with Listeriosis. *Journal of Food Protection* 78, 4 (2015), 738–745. doi:10.4315/0362-028X.JFP-14-527

[8] Yue Fan, Xiaojian Ma, Rongpeng Su, Jun Guo, Rujie Wu, Xi Chen, and Qing Li. 2025. *Embodied VideoAgent: Persistent Memory from Egocentric Videos and Embodied Sensors Enables Dynamic Scene Understanding.* arXiv:2501.00358 [cs] doi:10.48550/arXiv.2501.00358

[9] Yue Fan, Xiaojian Ma, Rujie Wu, Yuntao Du, Jiaqi Li, Zhi Gao, and Qing Li. 2025. VideoAgent: A Memory-Augmented Multimodal Agent for Video Understanding. In *Computer Vision – ECCV 2024*, Aleš Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol (Eds.). Vol. 15080. Springer Nature Switzerland, 75–92. doi:10.1007/978-3-031-72670-5_5

[10] Gabriele Goletto, Tushar Nagarajan, Giuseppe Averta, and Dima Damen. 2024. *AMEGO: Active Memory from Long EGOcentric Videos.* arXiv:2409.10917 [cs] doi:10.48550/arXiv.2409.10917

[11] Google. 2026. *Gemini API Media Resolution.* Google. https://ai.google.dev/gemini-api/docs/media-resolution Token accounting for Gemini media-resolution settings..

[12] Google. 2026. *Gemini API Pricing.* Google. https://ai.google.dev/gemini-api/docs/pricing Per-token input/output pricing for Gemini 3 Pro and related Gemini API models..

[13] Kristen Grauman, Andrew Westbury, Eugene Byrne, Zachary Chavis, Antonino Furnari, Rohit Girdhar, Jackson Hamburger, Hao Jiang, Miao Liu, Xingyu Liu, et al. 2022. Ego4d: Around the world in 3,000 hours of egocentric video. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 18995–19012.

[14] Kristen Grauman, Andrew Westbury, Lorenzo Torresani, Kris Kitani, Jitendra Malik, Triantafyllos Afouras, Kumar Ashutosh, Vijay Baiyya, Siddhant Bansal, Bikram Boote, et al. 2024. Ego-exo4d: Understanding skilled human activity from first-and third-person perspectives. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 19383–19400.

[15] Yifei Huang, Guo Chen, Jilan Xu, Mingfang Zhang, Lijin Yang, Baoqi Pei, Hongjie Zhang, Lu Dong, Yali Wang, Limin Wang, et al. 2024. Egoexolearn: A dataset for bridging asynchronous ego-and exo-centric view of procedural activities in real world. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 22072–22086.

[16] Zeyi Huang, Yuyang Ji, Xiaofang Wang, Nikhil Mehta, Tong Xiao, Donghyun Lee, Sigmund Vanvalkenburgh, Shengxin Zha, Bolin Lai, Licheng Yu, et al. 2025. Building a mind palace: Structuring environment-grounded semantic graphs for effective long video analysis with llms. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 24169–24179.

[17] Jaesung Huh, Jacob Chalk, Evangelos Kazakos, Dima Damen, and Andrew Zisserman. 2025. Epic-sounds: A large-scale dataset of actions that sound. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2025).

[18] Baoxiong Jia, Ting Lei, Song-Chun Zhu, and Siyuan Huang. 2022. Egotaskqa: Understanding human tasks in egocentric videos. *Advances in Neural Information Processing Systems* 35 (2022), 3343–3360.

[19] Zhaoyang Lv, Nicholas Charron, Pierre Moulon, Alexander Gamino, Cheng Peng, Chris Sweeney, Edward Miller, Huixuan Tang, Jeff Meissner, Jing Dong, Kiran Somasundaram, Luis Pesqueira, Mark Schwesinger, Omkar Parkhi, Qiao Gu, Renzo De Nardi, Shangyi Cheng, Steve Saarinen, Vijay Baiyya, Yuyang Zou, Richard Newcombe, Jakob Julian Engel, Xiaqing Pan, and Carl Ren. 2024. *Aria Everyday Activities Dataset.* arXiv:2402.13349 [cs.CV] doi:10.48550/arXiv.2402.13349

[20] Meta. 2023. *Introducing the New Ray-Ban Meta Smart Glasses.* Meta. https://about.fb.com/news/2023/09/new-ray-ban-meta-smart-glasses/

[21] Microsoft. 2026. *Azure OpenAI Service Pricing.* Microsoft. https://azure.microsoft.com/en-us/pricing/details/azure-openai/ Paid-tier input/output token pricing for Azure OpenAI Service..

[22] Microsoft. 2026. *Introducing a Powerful New Chapter for Windows PCs, Accelerated by NVIDIA RTX Spark.* Microsoft. https://blogs.windows.com/windowsexperience/2026/05/31/introducing-a-powerful-new-chapter-for-windows-pcs-accelerated-by-nvidia-rtx-spark/

[23] Matthias Minderer, Alexey A. Gritsenko, and Neil Houlsby. 2023. Scaling Open-Vocabulary Object Detection. *Advances in Neural Information Processing Systems* 36 (2023). https://papers.neurips.cc/paper_files/paper/2023/hash/e6d58fc68c0f3c36ae6e0e64478a69c0-Abstract-Conference.html

[24] Tushar Nagarajan, Yanghao Li, Christoph Feichtenhofer, and Kristen Grauman. 2020. Ego-topo: Environment affordances from egocentric video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 163–172.

[25] Tushar Nagarajan, Santhosh Kumar Ramakrishnan, Ruta Desai, James Hillis, and Kristen Grauman. 2023. EgoEnv: Human-centric environment representations from egocentric video. *Advances in Neural Information Processing Systems* 36 (2023), 60130–60143.

[26] NVIDIA. 2026. *NVIDIA and Microsoft Reinvent Windows PCs for the Age of Personal AI.* NVIDIA. https://nvidianews.nvidia.com/news/nvidia-microsoft-windows-pcs-agents-rtx-spark

[27] NVIDIA. 2026. *NVIDIA Jetson AGX Orin.* NVIDIA. https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/

[28] NVIDIA. 2026. *NVIDIA RTX 6000 Ada Generation.* NVIDIA. https://www.nvidia.com/en-us/products/workstations/rtx-6000/

[29] ONNX. 2026. *Introduction to ONNX.* ONNX. https://onnx.ai/onnx/intro/index.html

[30] OpenAI. 2026. *OpenAI API Pricing.* OpenAI. https://openai.com/api/pricing/ Per-token input/output pricing for GPT-5 family models..

[31] Rohith Peddi, Shivvrat Arya, Bharath Challa, Likhitha Pallapothula, Akshay Vyas, Bhavya Gouripeddi, Jikai Wang, Qifan Zhang, Vasundhara Komaragiri, Eric Ragan, Nicholas Ruozzi, Yu Xiang, and Vibhav Gogate. 2024. *CaptainCook4D: A Dataset for Understanding Errors in Procedural Activities.* arXiv:2312.14556 [cs] doi:10.48550/arXiv.2312.14556

[32] Toby Perrett, Ahmad Darkhalil, Saptarshi Sinha, Omar Emara, Sam Pollard, Kranti Parida, Kaiting Liu, Prajwal Gatti, Siddhant Bansal, Kevin Flanagan, Jacob Chalk, Zhifan Zhu, Rhodri Guerrier, Fahd Abdelazim, Bin Zhu, Davide Moltisanti, Michael Wray, Hazel Doughty, and Dima Damen. 2025. *HD-EPIC: A Highly-Detailed Egocentric Video Dataset.* arXiv:2502.04144 [cs] doi:10.48550/arXiv.2502.04144

[33] Chiara Plizzari, Gabriele Goletto, Antonino Furnari, Siddhant Bansal, Francesco Ragusa, Giovanni Maria Farinella, Dima Damen, and Tatiana Tommasi. 2024. An Outlook into the Future of Egocentric Vision: C. Plizzari et al. *International Journal of Computer Vision* 132, 11 (2024), 4880–4936.

[34] Will Price, Carl Vondrick, and Dima Damen. 2022. Unweavenet: Unweaving activity stories. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 13770–13779.

[35] Santhosh Kumar Ramakrishnan, Ziad Al-Halah, and Kristen Grauman. 2023. Naq: Leveraging narrations as queries to supervise episodic memory. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 6694–6703.

[36] Raspberry Pi Ltd. 2026. *Raspberry Pi Camera Module 3.* Raspberry Pi Ltd. https://www.raspberrypi.com/products/camera-module-3/

[37] Ray-Ban. 2026. *Ray-Ban Meta Smart Glasses Frequently Asked Questions.* Ray-Ban. https://www.ray-ban.com/usa/c/frequently-asked-questions-ray-ban-meta-smart-glasses

[38] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. 2016. You Only Look Once: Unified, Real-Time Object Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 779–788.

[39] Aniket Rege, Arka Sadhu, Yuliang Li, Kejie Li, Ramya Korlakai Vinayak, Yuning Chai, Yong Jae Lee, and Hyo Jin Kim. 2026. Agentic Very Long Video Understanding. *arXiv preprint arXiv:2601.18157* (2026).

[40] Ivan Rodin, Antonino Furnari, Kyle Min, Subarna Tripathi, and Giovanni Maria Farinella. 2024. Action scene graphs for long-form understanding of egocentric videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 18622–18632.

[41] Aditya Sharma, Michael Saxon, and William Yang Wang. 2024. Losing visual needles in image haystacks: Vision language models are easily distracted in short and long contexts. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 5429–5451.

[42] Xiaoqian Shen, Wenxuan Zhang, Jun Chen, and Mohamed Elhoseiny. 2026. Vgent: Graph-based retrieval-reasoning-augmented generation for long video understanding. *Advances in Neural Information Processing Systems* 38 (2026), 100805–100830.

[43] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Hervé Jégou, Patrick Labatut, and Piotr Bojanowski. 2025. DINOv3. arXiv:2508.10104 [cs.CV] https://arxiv.org/abs/2508.10104

[44] Yihong Sun, Xinyu Yang, Jennifer J. Sun, and Bharath Hariharan. 2025. *Tracking and Understanding Object Transformations.* arXiv:2511.04678 [cs] doi:10.48550/arXiv.2511.04678

[45] Masatoshi Tateno, Takuma Yagi, Ryosuke Furuta, and Yoichi Sato. 2025. Learning multiple object states from actions via large language models. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 9555–9565.

[46] Mahiro Ukai, Shuhei Kurita, and Nakamasa Inoue. 2025. STATUS Bench: A Rigorous Benchmark for Evaluating Object State Understanding in Vision-Language Models. In *Proceedings of the 33rd ACM International Conference on Multimedia*. 4718–4727.

[47] U.S. Bureau of Labor Statistics. 2025. American Time Use Survey—2024 Results. https://www.bls.gov/news.release/pdf/atus.pdf. USDL-25-1060; released June 26, 2025; accessed May 25, 2026.

[48] U.S. Department of Agriculture. 2021. New USDA Resources to Promote Reduction of Food Loss and Waste. https://www.usda.gov/about-usda/news/press-releases/2021/06/29/new-usda-resources-promote-reduction-food-loss-and-waste. Published June 29, 2021; accessed May 20, 2026.

[49] U.S. Food and Drug Administration. 2025. Food Loss and Waste. https://www.fda.gov/food/consumers/food-loss-and-waste. Content current as of January 13, 2025; accessed May 20, 2026.

[50] Weihan Wang, Zehai He, Wenyi Hong, Yean Cheng, Xiaohan Zhang, Ji Qi, Ming Ding, Xiaotao Gu, Shiyu Huang, Bin Xu, et al. 2025. Lvbench: An extreme long video understanding benchmark. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 22958–22967.

[51] Xin Wang, Taein Kwon, Mahdi Rad, Bowen Pan, Ishani Chakraborty, Sean Andrist, Dan Bohus, Ashley Feniello, Bugra Tekin, Felipe Vieira Frujeri, et al. 2023. Holoassist: an egocentric human interaction dataset for interactive ai assistants in the real world. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 20270–20281.

[52] Ziyang Wang, Honglu Zhou, Shijie Wang, Junnan Li, Caiming Xiong, Silvio Savarese, Mohit Bansal, Michael S Ryoo, and Juan Carlos Niebles. 2025. Active Video Perception: Iterative Evidence Seeking for Agentic Long Video Understanding. *arXiv preprint arXiv:2512.05774* (2025).

[53] Yuxuan Yan, Shiqi Jiang, Ting Cao, Yifan Yang, Qianqian Yang, Yuanchao Shu, Yuqing Yang, and Lili Qiu. 2025. AVA: Towards Agentic Video Analytics with Vision Language Models. *arXiv preprint arXiv:2505.00254* (2025).

[54] Parnian Zameni, Yuhan Shen, and Ehsan Elhamifar. 2025. Moscato: Predicting multiple object state change through actions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 11600–11611.

[55] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang, and Matthias Grundmann. 2020. MediaPipe Hands: On-device Real-time Hand Tracking. *arXiv preprint arXiv:2006.10214* (2020).

[56] Xiaoyi Zhang, Zhaoyang Jia, Zongyu Guo, Jiahao Li, Bin Li, Houqiang Li, and Yan Lu. 2025. Deep video discovery: Agentic search with tool use for long-form video understanding. *arXiv preprint arXiv:2505.18079* (2025).

[57] Yiyang Zhou, Yangfan He, Yaofeng Su, Siwei Han, Joel Jang, Gedas Bertasius, Mohit Bansal, and Huaxiu Yao. 2026. Reagent-v: A reward-driven multi-agent framework for video understanding. *Advances in Neural Information Processing Systems* 38 (2026), 151454–151491.

[58] Chenchen Zhu, Fanyi Xiao, Andrés Alvarado, Yasmine Babaei, Jiabo Hu, Hichem El-Mohri, Sean Culatana, Roshan Sumbaly, and Zhicheng Yan. 2023. Egoobjects: A large-scale egocentric dataset for fine-grained object understanding. In *Proceedings of the IEEE/CVF international conference on computer vision*. 20110–20120.

[59] Jialong Zuo, Yongtai Deng, Lingdong Kong, Jingkang Yang, Rui Jin, Yiwei Zhang, Nong Sang, Liang Pan, Ziwei Liu, and Changxin Gao. 2026. VideoLucy: Deep Memory Backtracking for Long Video Understanding. *Advances in Neural Information Processing Systems* 38 (2026), 25660–25691.